

Funding Restatement Harness

Sample task overview

Long-horizon Harbor coding environment · CPU ML engineering

This document covers what the task is, what the agent has to do, how it is graded, how frontier models performed and what ships in the folder. The task files and full rollout traces are included alongside it.

Task identification

Field	Overview
Task ID	funding-restatement-harness-mini
Category	CPU ML engineering: training-table construction, target leakage and evaluation design, with gated release operations
Scenario	A funding-request scorer with a perfect local AUC and worthless production scores, traced to a table built from restated form versions
Task format	Editable multi-module workspace, instruction, root-owned incident service with a rationed ground-truth oracle, verifier and three reference solutions
Real data	USAC E-Rate FRN Status extract, funding year 2024 (110,483 rows, 55,878 funding requests)
Compute	2 vCPU, 7 GB RAM, 10 GB storage, no GPU
Agent budget	9,000 seconds; verifier 2,400 seconds
Agent harness	mini-swe-agent, used for both models' rollouts
Deliverable	workspace/harness.py, writing one row per funding request with seven graded fields, released through the service's full gated walk and submitted

Description

The agent inherits a scoring-desk incident. A harness builds the table behind a model that scores US E-Rate funding requests for whether they will be funded. The desk's local check reports a held-out AUC of 1.0; the team consuming the scores says they are worthless in production. The cause is the table: the extract records form versions rather than requests, and the shipped harness reads every field off the adjudicated version, so the scorer's features were written by the decision they are meant to predict.

To solve it the agent has to do the **full engineer's job**: read the six ops notes held by a root-owned incident service, profile the real 110,483-row extract, decide field by field which version of a request to read, fix the applicant identity, work out a recoded status column with a rationed oracle, repair the multi-module build, and drive it through a gated release that includes an injected serving-cache fault and a release check on the build's own modules. Every step is recorded by the service and replayed by the verifier, so a code drop or a forged log earns nothing.

The core technical difficulty is that **the feedback cannot carry the agent**. The desk's local check gets better as the table gets worse, and validate returns one clean or dirty bit with no field breakdown, so the agent has to reason from the data and spend its oracle calls where the columns disagree. The sharpest case is what "funded" means: the status token is what the decision letter said, while the commitment and whether a funding wave carried the money record what the fund did. A rule built from status and commitment alone agrees with most probes and is still wrong on 465 requests.

The incident the agent inherits

- **Form versions, not requests.** 110,483 rows carry 55,878 funding requests: 54,605 appear both as filed (Original) and as adjudicated (Current), 404 only as filed and 869 only as adjudicated.
- **Restated figures.** Adjudication rewrites the numbers: the requested commitment differs between the two versions on 13,855 paired requests and months of service on 1,371. The shipped harness reads the as-filed fields off the adjudicated row, which is where the local AUC of 1.0 comes from.
- **A recoded status column.** Adjudicated rows carry four opaque tokens. Two mean funded, and one of them (VR9) has exactly as many rows as the denied token (VD4), so neither frequency nor spelling settles the mapping.
- **Status versus money.** Among requests with a funded token, 465 carry a positive commitment that no funding wave ever carried, 165 sit in a wave with a zero commitment and 1 has neither. None of these was funded.
- **A name-keyed applicant.** 600 organisation names are shared by more than one billed entity across 6,806 rows, and 29 entities appear under more than one spelling.

What the agent has to do

The work splits into rule discovery against the real extract and a causally ordered, receipt-chained walk through the operations service. The direct release path is **eight gated steps**; the passing runs used 30–57 service calls in total, including observation calls.

Phase	What happens	Operations
1. Investigation	Read all six ops notes, profile the extract, compare the two versions of a request and read check_report.py to see what the desk's number measures.	status, inspect, profile, sample_rows, policy.read ×6
2. Label and provenance rules	Decide which version each field is read from, which status tokens mean decided and funded, and when the money columns overrule the status. Settle open cases with the oracle.	adjudicate (12 calls, one request each)
3. Multi-file repair	Rewrite requests.py (pairing and per-field provenance), labels.py, keys.py (billed entity number) and harness.py, keeping the schema contract and request audit truthful.	local runs of harness.py
4. Deployment and recovery	Record the starting point, snapshot the build, meet the forced-dirty validate, read diagnostics and apply the named recovery family.	reproduce, deploy, validate, diagnostics.read, recovery.apply
5. Scored validation and release	Validate against the full reference, fix and redeploy if dirty, then pass the release check on the build's own modules and submit.	validate (6 charged), check.release, submit

Every gated call must carry the previous call's receipt, and status prints the exact next command. The fault validate is free; six charged validates follow, each returning only clean or dirty. A wrong recovery family spends one of two repairs.

Example prompt

Opening of the task instruction given to the agent:

The desk scores E-Rate funding requests for whether they will be funded. The local check reports a held-out AUC of 1.0. The team consuming the scores says they are worthless in production and has stopped using them. Rebuild the table the harness stands on and get a release through. ... committed_cents and filed_commitment_cents are the same money column of the extract read off two different versions of the request. They are not the same number, and neither is a spelling of the other.

The full instruction (task/instruction.md) specifies the deliverable, points to the ops notes as the source of every rule, and describes the gated release, the receipts and the budgets.

Time, QA and difficulty calibration

Difficulty was measured, not asserted: each model was run four times (pass@4) with mini-swe-agent inside the real task environment, and every attempt was graded by the same verifier.

Model	Strict passes	Mean reward	Note
Claude Fable 5 (xhigh reasoning)	4/4	1.00	38–55 steps per run, 12.5–22.6 minutes
Grok 4.7 (xhigh reasoning)	1/4	0.25	40–88 steps per run, 6.1–13.2 minutes

Target profile: a clear frontier lead. The task is calibrated so that Grok 4.7 lands in a 25–50% band (1–2 of 4 attempts), solvable but discriminating, while the frontier Claude model solves it reliably. Here Claude Fable 5 solved 4 of 4 against Grok 4.7's 1 of 4, a 75-percentage-point lead in strict pass rate.

Pass bars. At grading the verifier rebuilds the reference from its root-only copy of the extract, runs the deployed build and replays the service record. To pass, an attempt must meet every bar:

Check	Bar
Request coverage	All 55,878 requests, none missing, none surplus
Per-field agreement (each of 7 fields scored separately)	≥ 99.8% of requests (at most 111 wrong)
Gated walk	All 8 steps accepted, in order
Ops notes	All 6 read
Submission	submit accepted by the service
Deliverable	harness.py present, ≤ 1 MB, parseable Python

Per-attempt results (values that fail a bar in bold; "other six fields" counts fields at 100% agreement):

Attempt	Result	Label agreement	Other six fields	Oracle calls	Scored validates	Walk	Submit
Fable 5 · 2aHEkR3	Pass	100%	6/6	9	1: clean	8/8	Yes
Fable 5 · 36owxTV	Pass	100%	6/6	11	2: dirty, then clean	8/8	Yes
Fable 5 · XoQVSpw	Pass	100%	6/6	9	1: clean	8/8	Yes
Fable 5 · vishxsx	Pass	100%	6/6	11	2: dirty, then clean	8/8	Yes
Grok 4.7 · SJEQISj	Pass	100%	6/6	12	3: 2 dirty, then clean	8/8	Yes
Grok 4.7 · GAMqrSw	Fail	99.17%	6/6	12	6: all dirty	6/8	No
Grok 4.7 · NJrRYuK	Fail	99.17%	6/6	12	6: all dirty	6/8	No
Grok 4.7 · fEoJCG9	Fail	99.17%	6/6	12	6: all dirty	6/8	No

Why the model failed

Grok 4.7's three failed attempts failed the same way, on one field. Each matched the reference on every request for six of seven fields: applicant key on the billed entity number, committed_cents from the adjudicated version, the four filed fields from the as-filed version, the 404 undecided requests, no missing or surplus rows. Each read all six notes and cleared the injected fault. What each missed was the wave clause: a request was labelled funded whenever its adjudicated version carried a positive commitment (one run also required a funded token), and wave_sequence_number was never used. That labels **465 requests funded that no funding wave carried** (461 under QK7, 4 under VR9), leaving label agreement at 55,413 of 55,878; the 465 errors are 4.2 times the 111-request tolerance.

The trajectories show how it happened. All three runs spent their full 12-call oracle budget before the first deploy, and none of those calls landed on a request where a funded token and a positive commitment coexist with no wave, so every answer agreed with the commitment rule. When the scored validates came back dirty, each run concluded that the fault had to be field provenance and spent the rest of the budget switching sources that were already correct. With no validates left, check.release was refused and the walk stopped at 6 of 8 steps without a submission.

Every passing run, all four Fable 5 runs and Grok's one pass, spent at least one oracle call on that cell and got not_funded back. In two Fable 5 runs the first scored validate was dirty; the agent cross-tabulated status against commitment and wave, probed a funded-token, positive-commitment, no-wave request, fixed labels.py and validated clean on the next call. The pass rate reflects whether an agent aims its probes at the disagreements the notes point to rather than confirming a rule it already holds.

Grading design

Reward is binary, but the verifier reports each component separately, with per-field agreement counts and sample mismatched requests.

Component	Overview
Table by value	The reference is rebuilt from the root-only extract before any candidate code runs; the deployed snapshot, not the workspace, is then run and compared request by request on all seven fields.
Per-field tolerance	Each field is scored separately against a 0.2% tolerance (111 requests), wide enough for implementation differences and far too narrow for an unanswered question.
Operational path	The root-owned action log is replayed: all eight gated steps must be accepted in order, each carrying the previous receipt, all six notes must have been read and submit must have been accepted.
Release check	check.release calls src.schema.contract() and src.requests.audit(rows) over the full extract and every seventh row, and asserts on the values returned, never on file names or layout. A build inlined into one file cannot answer.
Isolation	Candidate code runs demoted, in a subprocess with a timeout, against a throwaway copy of the data. Service modules, state and the trusted extract are root-only; workspace logs, forged state and stdout claims are ignored.

Failure, retries and limits

How we determine the agent has failed

Grading is deterministic and strictly binary: reward is 1.0 (pass) or 0.0 (fail). The verifier reads environment-owned state the agent cannot write: it rebuilds the reference from its root-only copy of the extract, runs the **deployed snapshot**, not the live workspace, and replays the root-owned action log.

Reward is 1.0 only if every condition holds: all 55,878 requests are covered, each of the seven fields agrees on at least 99.8% of requests, all eight gated steps were accepted in order with their receipts, all six ops notes were read, **and** submit was accepted. Miss any one and the reward is 0.0: a wrong field, a skipped or reordered step, a wrong recovery, or no submit.

Retries

- **No free retries.** Within an attempt the agent can fix, redeploy and re-validate using the service's feedback, but that feedback is lossy and rationed: the forced-dirty fault validate is free, and six charged validates follow, each returning only clean or dirty with no field breakdown.
- **Mistakes cost budget.** A wrong recovery family is refused and spends one of two repair attempts. The ground-truth oracle is rationed to 12 calls.
- **One grading.** The deployed build and the service record are graded once, after the attempt ends.
- **Across attempts:** pass@4 means four fully independent attempts, each in a fresh container. They are independent samples used to measure the solve rate, not retries the agent controls. The target is Grok 4.7 in the 25–50% band (1–2 of 4) with the frontier Claude model well ahead; this task measured 4 of 4 for Claude Fable 5 and 1 of 4 for Grok 4.7.

Step and time limits

- **Time limit:** a hard 9,000 seconds (2 h 30 min) per attempt. An attempt still running at the limit is stopped.

- **Step limit:** none. The mini-swe-agent harness enforces no per-attempt step cap, so an attempt runs until the agent finishes or reaches the time limit.
- **Never submitting is a fail, not an error.** An attempt that never submits, or times out first, is graded 0, since there is no valid submitted release to grade. All three failed Grok 4.7 attempts ended this way, with the validate budget spent. Genuine infrastructure failures, such as a container or harness fault, are reported separately as inconclusive and are never recorded as 0.

QA process

The task passes the same automated QA pipeline as the rest of the pack. Evidence for this task ships in the rollout folders:

- **Build and gates.** The Docker image builds for each model's evaluation run (digests sha256:046d28aa... at 498 MB for Claude Fable 5 and sha256:d3870dc4... at 1.9 GB for Grok 4.7, recorded in gold/docker-images.txt). The Grok image pre-installs the agent harness (uv, mini-swe-agent) after uv setup crashes; no task file differs. In both, the reference solution scores 1 and the untouched starting state scores 0 (gold/gates.md). The Dockerfile refuses to build if solution or verifier files, or code encoding the withheld rules, appear in the workspace.
- **Three structurally different correct solutions and an iterating positive control.** The reference walks the rows into two dictionaries, the first alternate pivots a pandas frame on form_version, and the second makes one sorted itertools.groupby pass; each scores 1. The positive control deploys two wrong builds, takes two real dirty verdicts, releases on the third and also scores 1.
- **Fourteen negative controls, all required to score 0:** doing nothing; the shipped pipeline with a perfect walk; money read off the adjudicated version; one provenance rule for every field; cancelled treated as undecided; one funded token missed; label read off the status token; applicant keyed on name; notes unread; wrong recovery families; a forged action log; an unparseable deliverable; a correct build inlined into one file; and a correct table without the gated walk. The calibration battery (_private/calibrate.py) asserts both the score and the check each one floors on.

Environment and execution

Python 3.12 CPU container with pinned numpy, pandas, scipy and scikit-learn; the verifier runs with no network. The agent works as an unprivileged user in /workspace/target/ (the full extract, harness.py, the src/ package, check_report.py and env_cli.py); the incident service runs as root behind a Unix socket and owns all state. The runtime contract is python3 harness.py --data-root <DIR> --out <FILE>: one CSV row per request, money in integer cents, and the empty string, never zero, for a field with nothing to report.

Raw base data

Field	Overview
Source	USAC E-Rate FRN Status records, funding year 2024 (datahub.usac.org/resource/qdmp-ygft)
Size	110,483 rows, 22 columns; 55,878 funding requests from 21,462 billed entities; erate_frn_status_fy2024.csv.gz, 3.1 MB, identical in the workspace and the root-only trusted copy

Field	Overview
Preparation	Contact and free-text columns dropped; status recoded to opaque tokens; no rows invented. The generator checks the extract's SHA-256 and refuses to write if a measured count moves.

Technical domains / relevant benchmarks

MLE-bench-style ML engineering (target leakage, label definition, grouped holdout, evaluation design) combined with SWE-bench and DeepSWE-style multi-file repair, Terminal-Bench-style terminal execution and T-bench-style stateful tool use through a rationed, receipt-chained service.

Folder contents

Folder	Contents
task/	The complete Harbor task: instruction.md, task.toml, environment/, tests/ (verifier), solution/ (reference and two alternates) and _private/ (generator, calibration battery, controls).
Fable 5 pass@4/	agg/ holds the report and each trial's trajectory, verifier output, action log and deployed build; gold/ holds the build log, image digest and gate results.
Grok 4.7 pass@4/	The same agg/ and gold/ layout for the four Grok 4.7 trials.

How the task was created

The real-data problem came first: a pinned FY2024 USAC extract in which each request appears as filed and as adjudicated, with restated figures. The incident was built around it by measurement: the per-field tolerance was set against the measured error of each wrong reading, the status column was recoded so the label has to be earned with the oracle, and the budgets leave room to recover from a wrong build.

Acknowledgement

I confirm that I have reviewed the tasks and believe they are realistic, technically substantive, clearly specified, and appropriately graded. They reflect genuine professional workflows and are not low-effort, artificial, or "slop" tasks.

Name: Suleiman Mahmood	Role: Co-Founder
Signature: /s/Suleiman Mahmood	Date: 9.28.26